

MÓDULO 2 — PRÁCTICA

Test Management, Métricas y Reporting

Configuración de Allure Reports + Plantilla de Reporte Ejecutivo

Equipo: Riccetto y Gasco _____ Fecha: _____

PARTE 1 — Configuración de Allure Reports en TaskFlow

En esta sección configurarán Allure Reports en su proyecto TaskFlow para generar reportes HTML profesionales directamente desde su suite de tests de Vitest.

1.1 Instalación de dependencias

Ejecutar en la raíz del proyecto TaskFlow:

```
# Dependencias principales
npm install --save-dev allure-vitest allure-js-commons

# CLI de Allure para generar y visualizar el reporte
npm install --save-dev allure-commandline
```

allure-vitest es el reporter oficial para Vitest. allure-commandline permite generar el HTML desde la terminal sin instalar Java por separado.

1.2 Configuración en vite.config.ts

Modificar el archivo vite.config.ts en la raíz del proyecto para agregar el reporter de Allure:

```
// vite.config.ts
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    reporters: [
      'default', // mantiene la salida en terminal
      ['allure-vitest', { // agrega el reporter de Allure
        resultsDir: 'allure-results',
      }],
    ],
  },
  setupFiles: ['allure-vitest/setup'],
});
```

El directorio allure-results contendrá los archivos JSON intermedios. El HTML final se genera en un paso separado.

1.3 Scripts en package.json

Agregar los siguientes scripts para facilitar la generación y visualización del reporte:

```
// package.json
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest",
    "allure:generate": "allure generate allure-results --clean -o allure-report",
    "allure:open": "allure open allure-report",
    "allure:report": "npm run test && npm run allure:generate && npm run allure:open"
  }
}
```

1.4 Verificación: primer reporte

Ejecutar la suite de tests y generar el reporte para verificar que la configuración es correcta:

```
# Paso 1: ejecutar los tests (genera allure-results/)
npm test

# Paso 2: generar el HTML
npm run allure:generate

# Paso 3: abrir el reporte en el navegador
npm run allure:open
```

Se debería abrir el navegador con el dashboard de Allure mostrando los resultados de la suite actual. Verificar que aparecen los tests organizados por archivo.

✓ Checklist de verificación — Paso 1.4

☐	El directorio allure-results/ se creó después de npm test
☐	El comando allure:generate no dio errores
☐	Se abrió el navegador con el reporte HTML
☐	Se visualizan los tests en la vista Suites
☐	El pass rate en el Overview coincide con la terminal

PARTE 2 — Anotaciones: enriquecer el reporte con metadata

Las anotaciones convierten cada test en una pieza de documentación viviente: vinculan el test a una user story, le asignan severidad, y permiten navegar desde el reporte directamente al issue o PR en GitHub.

2.1 API de anotaciones disponibles

Anotación	Valores aceptados	Propósito
<code>allure.label('feature', ...)</code>	<i>'Autenticación', 'Proyectos', ...</i>	Agrupar tests en la vista Behaviors por feature
<code>allure.label('story', ...)</code>	<i>'US-01', 'US-02', ...</i>	Vincula el test a una user story dentro de la feature
<code>allure.severity(...)</code>	<i>blocker / critical / normal / minor / trivial</i>	Clasifica la criticidad del test — afecta priorización de defectos
<code>allure.link(url, name)</code>	<i>URL de GitHub issue, PR, o documentación</i>	Enlace navegable desde el reporte al issue relacionado
<code>allure.description(...)</code>	<i>Texto libre</i>	Descripción del objetivo del test — aparece en el detalle
<code>allure.step(...)</code>	<i>Texto libre</i>	Subdivide el test en pasos visibles en el reporte
<code>allure.attach(...)</code>	<i>Buffer / string + tipo MIME</i>	Adjunta screenshots, logs, respuestas JSON al resultado

2.2 Ejemplo completo: test con anotaciones

El siguiente ejemplo muestra un test de autenticación con todas las anotaciones aplicadas. Tomar como modelo para anotar los propios tests:

```
// src/auth/__tests__/auth.service.spec.ts
import { describe, it, expect, beforeEach } from 'vitest';
import { allure } from 'allure-vitest';
import { AuthService } from '../auth.service';

describe('AuthService - Bloqueo de cuenta', () => {

  it('bloquea la cuenta tras 5 intentos fallidos', async () => {
    // Anotaciones de trazabilidad
    allure.label('feature', 'Autenticación');
    allure.label('story', 'US-03');
    allure.severity('critical');
    allure.link('https://github.com/org/taskflow/issues/47', 'BUG-05');
    allure.description(
      'Verifica que la cuenta se bloquea exactamente al 5to intento. ' +
      'Relacionado con BUG-05: off-by-one en auth.service.ts (> 5 → >= 5).'
    );

    const service = new AuthService();

    // Paso 1: realizar 4 intentos fallidos
```

```

await allure.step('4 intentos fallidos previos', async () => {
  for (let i = 0; i < 4; i++) {
    await service.login('user@test.com', 'wrongpassword');
  }
});

// Paso 2: verificar bloqueo en el 5to intento
await allure.step('5to intento - debe lanzar error de bloqueo', async () => {
  await expect(
    service.login('user@test.com', 'wrongpassword')
  ).rejects.toThrow('Account locked');
});
});
});

```

El uso de `allure.step()` es especialmente útil en tests E2E y de integración donde el flujo tiene múltiples etapas. En tests unitarios simples puede omitirse.

2.3 Actividad: anotar 5 tests del proyecto TaskFlow

Seleccionar 5 tests existentes en el proyecto y aplicar las anotaciones correspondientes. Completar la siguiente tabla de registro:

Archivo / Test	feature	story	severity	link (issue o PR)
1.auth.service.spec.ts/ acepta email válido	Autenticación	US-01	normal	https://github.com/JMRicetto/taskflow/pull/1
2.auth.service.spec.ts/ rechaza contraseña menor a 8	Autenticación	US-01	normal	https://github.com/JMRicetto/taskflow/pull/2
3.auth.service.spec.ts/ etorna token para credenciales	Autenticación	US-02	critica	https://github.com/JMRicetto/taskflow/pull/3
4.auth.service.spec.ts/ bloquea la cuenta después de 5	Autenticación	US-02	critica	https://github.com/JMRicetto/taskflow/pull/5
5.task.service.spec.ts/ lanza error si título < 3	Tareas	US-04	normal	https://github.com/JMRicetto/taskflow/pull/4

2.4 Regenerar el reporte y verificar la vista Behaviors

Después de aplicar las anotaciones, ejecutar nuevamente:

```
npm test && npm run allure:generate && npm run allure:open
```

Allure

Overview

Categories

Suites

Behaviors

Packages

Graphs

Timeline

Behaviors

order ↑↓ name ≡ duration ↑↓ status ↑↓

Status: 006700

Marks:     

Autenticación

US-01

#1 acepta email válido203ms

#2 rechaza contraseña menor a 8 caracteres4s 673ms

US-02

#2 bloquea la cuenta después de 5 intentos fallidos—BUG-05395ms

#1 retorna token para credenciales válidas395ms

Tareas

US-04

#1 lanza error si el título tiene menos de 3 caracteres1ms

✔ Checklist de verificación — Parte 2

☐	Los 5 tests anotados aparecen en la vista Behaviors agrupados por feature
☐	La vista Overview muestra el breakdown por severity (critical, normal, etc.)
☐	Al hacer click en un test, se visualizan los pasos de allure.step()
☐	El enlace al issue/PR es navegable desde el reporte
☐	El reporte se genera sin errores con los datos de cobertura actuales

TaskFlow Project · Reporte Ejecutivo de Testing

PARTE 3 — Plantilla de Reporte Ejecutivo de Testing

Completar esta plantilla con los datos reales del proyecto TaskFlow. El objetivo es producir un Completion Report de 1 página legible por stakeholders no técnicos.

Usar los datos del reporte de Allure generado en la Parte 2 y otros datos que puedan obtener del proyecto.

REPORTE EJECUTIVO DE TESTING

Testing y Calidad de Software 2026 · TaskFlow Project

Equipo / Proyecto TaskFlow — Joaquín Gasco y Juan Riccetto	Release / Sprint ej. Sprint 14 · Release 1.0	Período cubierto ej. 01/05 – 28/05/2026	Fecha del reporte ej. 01/06/2026
--	--	---	--

A. Estado general del ciclo de testing

Área evaluada	 OK	 Riesgo	 Bloqueante
Cobertura de código alcanza el umbral (≥ 80%)	x	☐	☐
Pipeline CI/CD verde en la rama principal	x	☐	☐
Sin defectos críticos abiertos sin asignar	x	☐	☐
Tests E2E ejecutan sin flaky failures	x	☐	☐
Contract tests (Pact) consumer + provider verificados	x	☐	☐

B. Métricas clave del ciclo

Métrica	Valor obtenido	Umbral / Meta	Interpretación
Pass Rate	100%	≥ 95%	Todos los tests unitarios, de integración, Pact, BDD y E2E se ejecutan con éxito
Cobertura de código	94,44%	≥ 80%	Supera ampliamente la meta mínima
Defect Density (módulo crítico)	0 / KLOC	< 3 / KLOC	Densidad de defectos nula ya que todos los bugs conocidos se corrigieron y validaron en este ciclo
DDP (Defect Detection %)	100%	≥ 95%	Alta efectividad de detección en la suite de pruebas previa a release
Escaped Defects	0	0	No se registraron defectos escapados a producción durante este sprint

Métrica	Valor obtenido	Umbral / Meta	Interpretación
Fix Rate (sprint actual)	100%	100%	Corrección completa al 100% de los defectos identificados en el ciclo de pruebas

C. Resumen de defectos

Total defectos encontrados 3	Defectos corregidos 3 (100%)	Defectos diferidos 0	Escaped defects 0 en producción
---------------------------------	---------------------------------	-------------------------	------------------------------------

Defecto más crítico del ciclo y causa raíz identificada:

Descripción del defecto: El backend no bloqueaba la cuenta del usuario exactamente al alcanzar el límite establecido de intentos de autenticación fallidos. Causa raíz: Error de lógica condicional en <code>`auth.service.ts`</code> donde se realizaba una validación mayor estricta (<code>`failedLogins > 5`</code>) en lugar de mayor o igual (<code>`failedLogins >= 5`</code>) para disparar la penalización temporal.

D. Evaluación del riesgo residual

Área / Módulo	Probabilidad	Impacto	Medida de mitigación
Autenticación y Sesiones	Bajo	Alto	Verificación estricta de expiración de JWT en cada petición
Base de Datos / Persistencia	Bajo	Alto	Uso de migraciones controladas por Prisma y aislamiento completo de transacciones en base de datos
Desempeño bajo carga	Medio	Medio	Monitoreo activo de latencias y optimizaciones

E. Deuda técnica identificada en tests

Tipo de deuda	Archivo / Módulo	Prioridad de pago	Acción propuesta
Tests unitarios incompletos	task.service.ts	media	Incrementar cobertura en flujos alternativos del método <code>`createTask`</code>
Flaky Tests / Tiempos de espera	E2E con Playwright	baja	Utilizar localizadores semánticos estables y aserciones dinámicas en lugar de timeouts fijos para evitar fragilidad en diferentes entornos

Tipo de deuda	Archivo / Módulo	Prioridad de pago	Acción propuesta
Mocks complejos en base de datos	`auth.service.spec.ts`	baja	Refactorizar y modularizar fixtures reutilizables para simplificar el mantenimiento de las pruebas unitarias

F. Conclusión y recomendación de release

Decisión: ☒ GO — apto para release ☐ NO-GO — requiere correcciones ☐ GO con observaciones

Justificación (2–3 oraciones basadas en las métricas):

El release de TaskFlow cumple rigurosamente con todos los estándares y umbrales de calidad establecidos. Se ha alcanzado un 100% de éxito en la ejecución de la suite completa de pruebas (Unitarias, Integración, Pact, BDD y E2E con Playwright) y una cobertura de código de 94.44% (excediendo ampliamente la meta del 80%).

Recomendaciones para el próximo ciclo:

- Integrar pruebas de rendimiento automatizadas de latencia en la pipeline de CI/CD.
- Implementar alertas automáticas de intentos de login fallidos en el entorno productivo para mitigar ataques de fuerza bruta.

REFERENCIA RÁPIDA — Comandos Allure

Comando	Qué hace
<code>npm test</code>	Ejecuta la suite y genera allure-results/ con archivos JSON
<code>npm run allure:generate</code>	Transforma los JSON en un reporte HTML en allure-report/
<code>npm run allure:open</code>	Abre el reporte HTML en el navegador
<code>npm run allure:report</code>	Los 3 pasos anteriores en un solo comando
<code>npx allure serve allure-results</code>	Alternativa: genera y abre sin guardar allure-report/
<code>rm -rf allure-results allure-report</code>	Limpiar resultados anteriores antes de una nueva corrida

Agregar allure-results/ y allure-report/ al .gitignore del proyecto para no versionar los reportes generados localmente.

Troubleshooting común

Problema	Solución
allure-results/ no se genera	Verificar que setupFiles: ['allure-vitest/setup'] está en vite.config.ts
allure command not found	Usar npx allure generate ... o agregar ./node_modules/.bin al PATH
El reporte está vacío	Los tests deben haber corrido con vitest run, no vitest (modo watch)
Error: Cannot find module 'allure-vitest'	Ejecutar npm install --save-dev allure-vitest allure-js-commons
Los behaviors no aparecen	Verificar que allure.label('feature', ...) está dentro del cuerpo del test, no en describe()